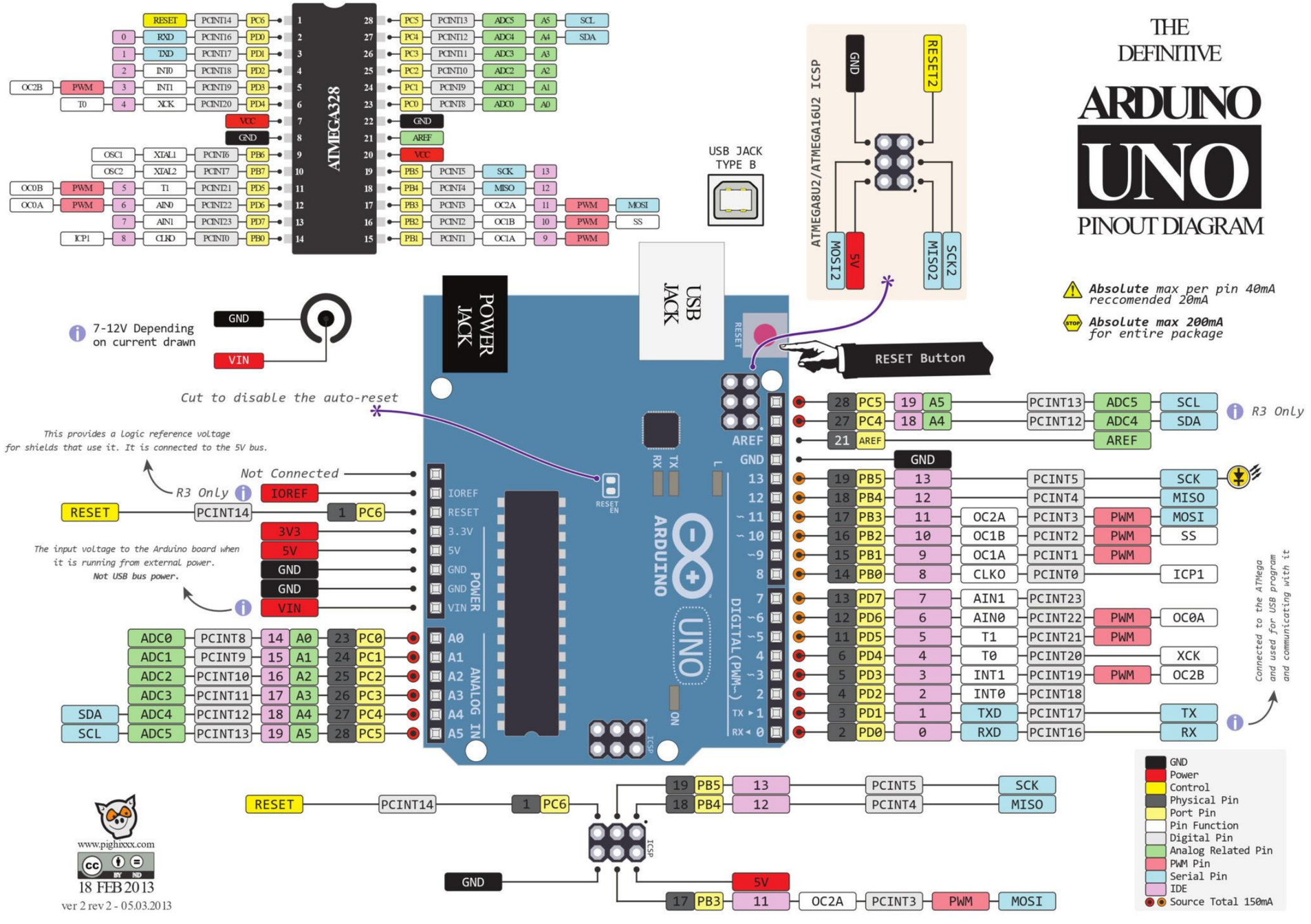
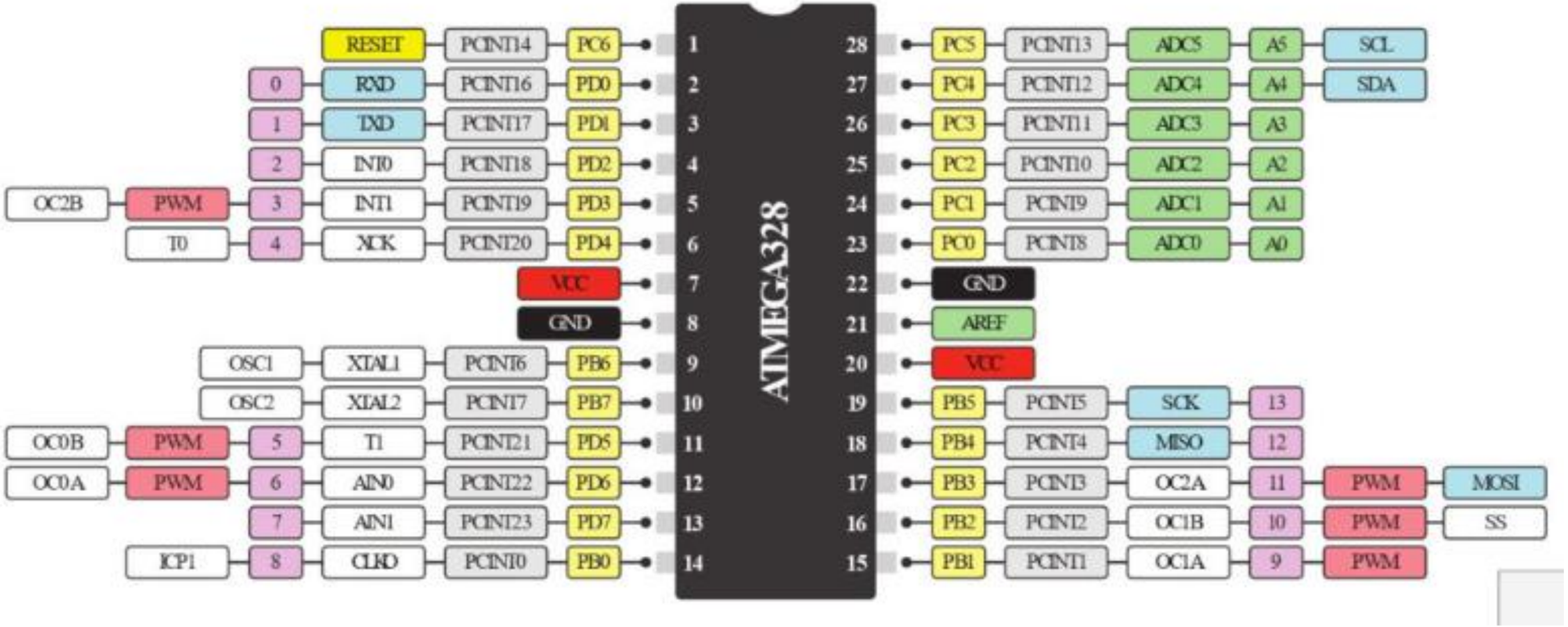
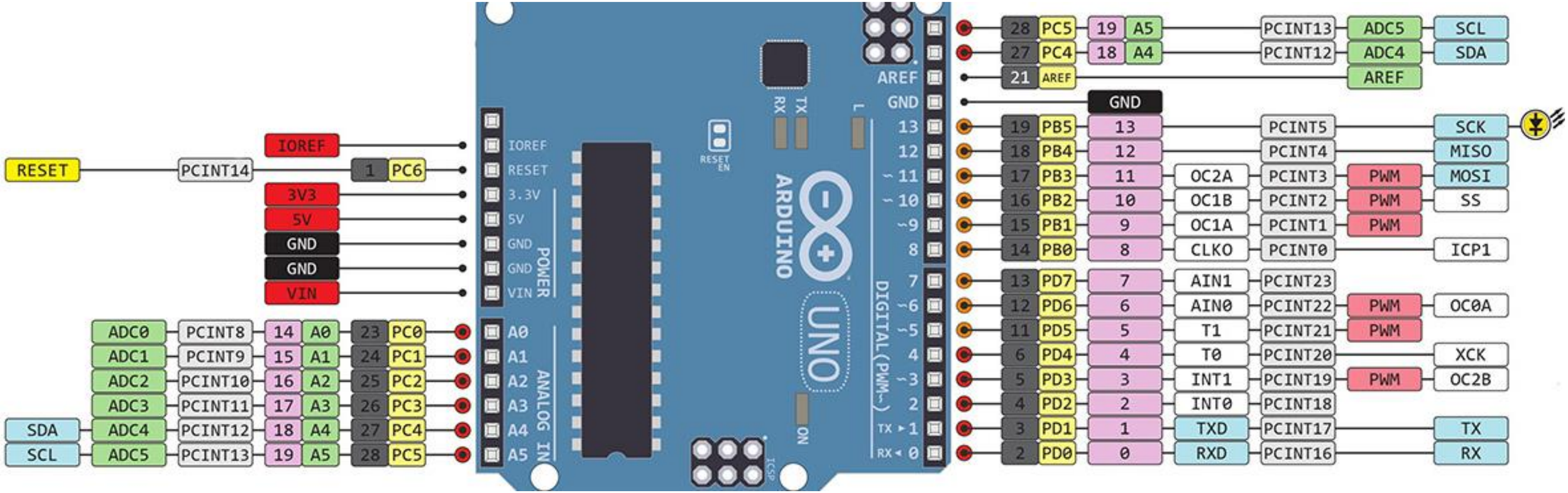
Arduino Uno Pinout Diagram



ATmega328 Pinout Diagram



Arduino Uno Pinout Diagram



Problem 1: Blink Program using Timer

An Arduino microcontroller code is to be written for a 600 ms interval LED blinking system using TIMERn Interrupt and a Pre-scalar value of 1024. LED is connected to pin 5 of the board.
Compute the value to be loaded into the OCRnB register. The system clock frequency is 16 MHz. The Timer ISR vector is TIMERn_COMPx_vect. [n = 0-2; x = A/B]. Determine the contents of the following program.

```
bool LED_State = HIGH;

void setup() {
    pinMode(5, OUTPUT);
    cli();
    TCCR__A = 0b_____________;          |COMnA1|COMnA0| COMnB1| COMnB0| - | - |WGMn1|WGMn0|
    TCCR__B = 0b_____________;          |FOCnA|FOCnB| - | - |WGMn2|CSn2| CSn1| CSn0|
    TCCR__B = 0b_____________;
    TIMSK___ = 0b_____________;         | - | - | ICIEn | - | - |OCIEnB|OCIEnA|TOIEn|
    OCR_____ = ___________________;     | - | - | - | - | - | - | - | - |
    sei();
}

void loop() {
    // main code here, to run repeatedly.
}

ISR(TIMER____COMP____vect) {
            TCNT__ = 0x_____________________;
            LED_State = !LED_State;
            DigitalWrite(_______, __________);
}
```

Solution to Problem 1

System Clock frequency, $f_{sys\_clk} = 16$ MHz $= 16 \times 10^6$ Hz;

Timer Clock frequency, $f_{timer\_clk} = f_{sys\_clk}/1024 = 16 \times 10^6$ Hz$/1024 = 15625$ Hz;

Timer Clock Period, $T_{timer\_clk} = 1/f_{timer\_clk} = 1/(15625 \text{ Hz}) = 64 \times 10^{-6}$ s

Required Delay $= 600$ ms $= 600 \times 10^{-3}$ s

$$\text{Timer count} = \frac{\text{Required delay}}{\text{Timer Clock Period}} - 1 = \frac{600 \text{ x } 10{-}3 \text{ s}}{64 \text{ x } 10{-}6 \text{ s}} - 1 = 9374$$

Timer0 and Timer2 can count up to 255, whereas Timer1 can count up to 65,535; therefore, only Timer1 is suitable for this application. So, the Timer Count value (9374) should be loaded into the OCR1B register.

Solution to Problem 1 (Contd..)

```
volatile bool LED_State = HIGH;

void setup() {
    pinMode(5, OUTPUT);
    cli();
    TCCR1A = 0b00000000;      |COMnA1|COMnA0| COMnB1| COMnB0| - | - |WGMn1|WGMn0|
    TCCR1B = 0b00000000;      |FOCnA|FOCnB| - | - |WGMn2|CSn2| CSn1| CSn0|
    TCCR1B = 0b00000101;
    TIMSK1 = 0b00000100;      | - | - | - | - | - |OCIEnB|OCIEnA|TOIEn|
    OCR1B = 9374;             | - | - | - | - | - | - | - | - |
    sei();
}

void loop() {
    // nothing is added here as interrupt handles the blinking operation
}

ISR(TIMER1_COMPB_vect){
    TCNT1 = 0x0000;
    LED_State = !LED_State;
    digitalWrite(5, LED_State);
}
```

Problem 2

Write an Arduino microcontroller code for a 15 ms interval LED blinking system using TIMERn Interrupts and a Pre-scalar value of 1024. LED is connected to pin 5 of the board. Compute the value to be loaded into the OCRnA register. [n = 0-2].

Solution to Problem 2

System Clock frequency, f_{sys_clk} = 16 MHz = 16 x 10^6 Hz;

Timer Clock frequency, $f_{timer\_clk} = f_{sys\_clk}$ /1024 = 16 x 10^6 Hz/1024 = 15625 Hz;

Timer Clock Period, $T_{timer\_clk} = 1/ f_{timer\_clk}$ = 1/(15625 Hz) = 64 x 10^{-6} s

Required Delay = 15 ms = 15 x 10^{-3} s

$$\text{Timer count} = \frac{\text{Required delay}}{\text{Timer Clock Period}} - 1 = \frac{15 \text{ x } 10{-}3 \text{ s}}{64 \text{ x } 10{-}6 \text{ s}} - 1 = 234.375 - 1 = 233.375 \cong 233$$

Timer0 and Timer2 can count up to 255, whereas Timer1 can count up to 65,535; therefore, any Timer is suitable for this application. So, the Timer Count value (233) should be loaded into the OCRnA register.

Solution to Problem 2 (Contd..)

The program for Arduino is as follows:

```
volatile bool LED_State = HIGH;          // Declaring LED_State as a volatile bool variable

void setup() {
    pinMode(5, OUTPUT);                  // Setting pin 5 of Arduino Uno as Output
    cli();                               // Disabling all interrupts until the setting is complete
    TCCR0A = 0;                          // Resetting the entire A and B registers of Timer0 to make sure that
    TCCR0B = 0;                          // everything is clear, and mode is set to normal.
    TCCR0B = 0b00000101;                 // Setting CS12:10 bits of TCCR0B to 101 to get a pre-scalar value of 1024
    TIMSK0 = 0b00000010;                 // Setting OCIE0A bit to 1 to enable the compare match A interrupt
    OCR0A = 233;                         // Setting compare match register, A for 15ms interval
    TCNT0  = 0;                          // Initializing counter value to 0
    sei();                               // Enabling global interrupt bit i.e. enabling all interrupts
}

void loop() {
    // nothing is added here as interrupt handles the blinking operation
}

// With the settings above, this ISR will trigger in every 15 ms.
ISR(TIMER0_COMPA_vect) {
    TCNT0 = 0;                           // Restting counter value to 0 to make it ready for next interrupt
    LED_State = !LED_State;              // Inverting the LED State
    digitalWrite(5, LED_State);          // Writing this new state to the LED connected to pin 5
}
```

Problem 3

Write the code for Arduino Uno board that will make an LED connected to pin 8 toggle ON and OFF each time the button 2 is pressed. Use External interrupt INTn (n=0/1) for this.

Solution to Problem 3 using built-in function attachInterrupt ()

In Arduino Uno, external interrupt 0 and 1 correspond to digital pins 2 and 3 respectively.

```
2  External Interrupt Request 0  (pin D2)         (INT0_vect)
3  External Interrupt Request 1  (pin D3)         (INT1_vect)
```

```
volatile bool led_State = false;                // Declaring ledState as a volatile bool variable

// Defining ISR (Interrupt Service Routine) toggleLED
void toggleLED() {
    led_State = !led_State;                     // toggling the state of the LED
    digitalWrite(8, led_State);                 // Writing the state of LED to pin 8
}

// Initial Setting
void setup() {
    pinMode(8, OUTPUT);                         // Setting Pin 8 as a LED output pin
    pinMode(2, INPUT_PULLUP);                   // Setting Pin 2 as a Button input with internal pull-up resistor

    attachInterrupt(digitalPinToInterrupt(2), toggleLED, RISING);
                                                // run the function toggleLED when pin 2 detects a rising edge
}

void loop() {
    // Nothing is added here as  interrupt handles LED toggling
}
```

Alternative Solution to Problem 3

In Arduino Uno, external interrupt 0 and 1 correspond to digital pins 2 and 3 respectively.

```
2  External Interrupt Request 0  (pin D2)         (INT0_vect)
3  External Interrupt Request 1  (pin D3)         (INT1_vect)
```

```
volatile bool led_State = false;              // Declaring ledState as a volatile bool variable

// Defining ISR (Interrupt Service Routine)
ISR (INT0_vect) {
    led_State = !led_State;                   // toggling the state of the LED
    digitalWrite(8, led_State);               // Writing the state of LED to pin 8
}

// Initial Setting
void setup() {
    pinMode(8, OUTPUT);                       // Setting Pin 8 as a LED output pin
    pinMode(2, INPUT_PULLUP);                 // Setting Pin 2 as a Button input with internal pull-up resistor

    cli();                                    // Disabling global interrupt
    EICRA = 0b00000011;                       // Setting INT0 to trigger on rising edge
    EIMSK = 0b00000001;                       // Enabling External Interrupt INT0
    sei();                                    // Enabling global interrupt
}

void loop() {
    // Nothing is added here as  interrupt handles LED toggling
}
```

Bit	7	6	5	4	3	2	1	0	
(0x69)	–	–	–	–	ISC11	ISC10	ISC01	ISC00	EICRA
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
0x1D (0x3D)	–	–	–	–	–	–	INT1	INT0	EIMSK
Read/Write	R	R	R	R	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	